

Herança, Classes Abstratas e Polimorfismo

Reaproveitando código e generalizando comportamentos em Java

Linguagem de Programação II

- Entender por que duplicar código entre classes parecidas é um problema.
- Usar **herança** (`extends`) para reaproveitar atributos e métodos.
- Entender a diferença entre *herdar* um atributo e *acessar* um atributo privado diretamente.
- Sobrescrever métodos com `@Override` e reaproveitar comportamento com `super`.
- Entender e aplicar **polimorfismo**: tratar objetos diferentes de forma uniforme.
- Criar e usar **classes e métodos abstratos**.
- Conhecer as **definições formais** de herança, reuso, sobrescrita, polimorfismo e classe abstrata.

Parte 1 – Introdução

Por que precisamos de herança?

Contexto: Sistema de Funcionários de uma Loja

Uma loja precisa organizar as informações dos seus profissionais.

A equipe é formada por:

- **Atendentes** – atendem os clientes;
- **Gerentes** – coordenam as equipes;
- **Supervisores** – acompanham os setores;
- **Vendedores** – realizam as vendas.

Todos trabalham na mesma loja e compartilham várias informações.

Todos os profissionais possuem os mesmos atributos:

- nome;
- matrícula;
- salário base;
- setor;
- horas trabalhadas.

Todos precisam das mesmas operações:

- registrar horas trabalhadas;
- exibir os dados do profissional.

Uma primeira solução

Uma primeira ideia é criar uma classe separada para cada função da loja: Atendente, Gerente, Supervisor e Vendedor.

Cada classe precisaria ter:

- os atributos nome, matrícula, salário base, setor e horas trabalhadas;
- um construtor para inicializar esses atributos;
- métodos get e set;
- um método `exibirDados()`.

Implementação: classe Atendente

```
public class Atendente {  
    private String nome;  
    private String matricula;  
    private double salarioBase;  
    private String setor;  
    private int horasTrabalhadas;  
  
    public Atendente(String nome, String matricula,  
                     double salarioBase, String setor) {  
        this.nome = nome;  
        this.matricula = matricula;  
        this.salarioBase = salarioBase;  
        this.setor = setor;  
    }  
  
    public void exibirDados() {  
        System.out.println("Nome: " + nome);  
        System.out.println("Matricula: " + matricula);  
        System.out.println("Salario base: " + salarioBase);  
        System.out.println("Setor: " + setor);  
    }  
}
```

Implementação: classe Gerente

```
public class Gerente {
    private String nome;
    private String matricula;
    private double salarioBase;
    private String setor;
    private int horasTrabalhadas;

    public Gerente(String nome, String matricula,
                   double salarioBase, String setor) {
        this.nome = nome;
        this.matricula = matricula;
        this.salarioBase = salarioBase;
        this.setor = setor;
    }

    public void exibirDados() {
        System.out.println("Nome: " + nome);
        System.out.println("Matricula: " + matricula);
        System.out.println("Salario base: " + salarioBase);
        System.out.println("Setor: " + setor);
    }
}
```

Observação

As classes `Atendente` e `Gerente` possuem exatamente os mesmos atributos, o mesmo construtor e o mesmo método `exibirDados()`.

A única diferença entre elas, até agora, é **o nome da classe**.

O mesmo aconteceria se criássemos as classes `Supervisor` e `Vendedor`: todo esse código precisaria ser copiado novamente.

O problema da primeira solução

Ao criar uma classe independente para cada função, o mesmo código é escrito repetidamente:

- os atributos comuns aparecem em todas as classes;
- o construtor é praticamente idêntico em todas;
- os métodos get e set são repetidos;
- o método `exibirDados()` é copiado em cada classe.

Consequência

Se um atributo comum precisar mudar, será necessário alterar **a mesma coisa em várias classes diferentes**.

Crescendo o problema

A loja decide contratar um novo tipo de profissional: o **estoquista**.

Para isso, seria necessário criar mais uma classe e copiar novamente:

- os mesmos atributos;
- o mesmo construtor;
- os mesmos métodos get e set;
- o mesmo método `exibirDados()`.

Quanto mais funções a loja tiver, mais código repetido o sistema terá.

Como evitar essa repetição?

Como podemos escrever os atributos e métodos comuns **uma única vez** e reaproveitá-los em todas as classes de profissionais, sem copiar e colar o mesmo código?

Precisamos de um mecanismo que permita:

- definir as características comuns em um único lugar;
- reaproveitar esse código em Atendente, Gerente, Supervisor e Vendedor;
- permitir que cada classe continue existindo de forma independente, sem repetir o que já é comum.

Reuso de código

É a prática de escrever uma funcionalidade **uma única vez** e utilizá-la em múltiplos contextos, evitando duplicação e facilitando a manutenção.

- Já vimos uma forma de reuso: a **composição** (uma classe *tem um* atributo de outra classe, como `Cliente` e `Endereco`).
- Hoje vamos conhecer outra forma de reuso: a **herança**.

A solução: herança

- Criamos uma **superclasse** (classe-mãe) com tudo que é comum.
- As **subclasses** (classes-filhas) herdam esses atributos e métodos, e adicionam o que for específico.
- Relação "**é um**": um Cachorro é um Animal; um Gato é um Animal.
- Em Java, usamos a palavra-chave `extends`.

Herança

É um mecanismo da orientação a objetos em que uma classe (**subclasse**) adquire atributos e métodos de outra classe (**superclasse**), podendo reaproveitá-los e estendê-los com novos comportamentos.

- Palavra-chave em Java: `extends`.
- Representa sempre uma relação "**é um**" (*is-a*).

Definição: Superclasse e Subclasse

Superclasse (classe-mãe)

Classe mais **geral**, que concentra os atributos e métodos comuns a várias classes relacionadas.

Subclasse (classe-filha)

Classe mais **específica**, que herda da superclasse e adiciona (ou sobrescreve) o que for particular a ela.

Sintaxe básica da herança

```
public class Subclasse extends Superclasse {  
    // atributos e metodos extras  
}
```

- Em Java, uma classe só pode ter **uma** superclasse direta (não existe herança múltipla de classes).
- A subclasse herda automaticamente os métodos public (e protected) da superclasse.

Parte 2 – Herança na Prática

`extends`, `super()` e atributos privados

Exemplo: Superclasse Animal

```
public class Animal {  
    private String nome;  
    private int idade;  
  
    public Animal(String nome, int idade) {  
        this.nome = nome;  
        this.idade = idade;  
    }  
  
    public String getNome() { return nome; }  
    public int getIdade() { return idade; }  
  
    @Override  
    public String toString() {  
        return "Animal: " + nome + " | Idade: " +  
            idade;  
    }  
}
```

Atenção

- Mesmo um Cachorro sendo um `Animal`, ele **não pode** acessar `nome` ou `idade` diretamente – eles são `private` em `Animal`.
- Para inicializá-los, a subclasse usa `super(...)` dentro do seu construtor.
- Para lê-los, usamos os *getters* herdados: `getNome()`, `getIdade()`.
- Existe o modificador `protected`, que permite acesso direto das subclasses – mas vamos manter `private` + *getters*, seguindo o mesmo padrão de encapsulamento das aulas anteriores.

Exemplo: Subclasse Cachorro extends Animal

```
public class Cachorro extends Animal {  
    private String raca;  
  
    public Cachorro(String nome, int idade, String  
        raca) {  
        super(nome, idade);  
        this.raca = raca;  
    }  
  
    public String getRaca() { return raca; }  
  
    @Override  
    public String toString() {  
        return super.toString() + " | Raca: " + raca;  
    }  
}
```

Exemplo: usando Cachorro no Main

```
public class Main {  
    public static void main(String[] args) {  
        Cachorro rex = new Cachorro("Rex", 3, "  
            Labrador");  
  
        System.out.println(rex.getNome());  
        System.out.println(rex.getRaca());  
        System.out.println(rex);  
    }  
}
```

Saída:

```
Rex  
Labrador  
Animal: Rex | Idade: 3 | Raca: Labrador
```

O que a subclasse herda automaticamente

- Métodos e atributos `public` e `protected` são herdados pela subclasse.
- Métodos e atributos com acesso *default* (*package-private*) são herdados e acessíveis quando a subclasse está no mesmo pacote da superclasse.
- Membros `private` não podem ser acessados diretamente pela subclasse. Eles continuam fazendo parte do estado interno do objeto e devem ser utilizados por meio dos métodos disponibilizados pela superclasse.
- O **construtor não é herdado**: a subclasse precisa declarar seu próprio construtor e pode chamar o construtor da superclasse com `super(...)`.
- Se `super(...)` não for escrito explicitamente, o Java tenta inserir uma chamada a `super()`. Ocorrerá erro de compilação se a superclasse não possuir um construtor sem parâmetros acessível.

Herança vs. Composição

- **Herança** representa uma relação "**é um**": Cachorro *é um* Animal.
- **Composição** representa uma relação "**tem um**": Cliente *tem um* Endereco.
- Nem tudo deve virar herança – às vezes a composição é a escolha mais natural.
- Regra prática: antes de usar extends, pergunte-se: "*é um*" ou "*tem um*"?

Vantagens:

- **Reuso de código:** escrevemos uma vez na superclasse, usamos em várias subclasses.
- **Organização:** a hierarquia reflete o domínio do problema.
- **Manutenção:** corrigir algo na superclasse corrige em todas as subclasses.

Cuidados:

- Hierarquias muito profundas ficam difíceis de entender.
- Herança cria um acoplamento forte entre superclasse e subclasse.
- Use herança apenas quando existir, de fato, uma relação "é um".

Em poucas palavras

- `extends` cria uma relação "**é um**" entre subclasse e superclasse.
- Atributos privados continuam privados; usamos `super(...)` e *getters*.
- O construtor não é herdado – cada subclasse cria o seu, chamando `super(...)`.
- Herança é uma forma de **reuso de código**, assim como a composição.

Precisamos implementar o som de cada animal

- Até agora, a classe `Animal` não possui nenhum método para representar o som que o animal emite.
- Sabemos que cada tipo de animal emite um som diferente: o cachorro late, o gato mia.

Pergunta

Onde devemos implementar o método `emitirSom()`? Em cada subclasse separadamente, ou em algum lugar comum a todas?

Colocando emitirSom() em Animal

- Como já aprendemos com herança, se colocarmos o método em `Animal`, **todas** as subclasses o herdam automaticamente – sem precisar reescrevê-lo em cada uma.
- Vamos então implementar em `Animal` uma versão genérica de `emitirSom()`.

Mas surge um problema

Se Cachorro e Gato apenas herdarem esse método sem nenhuma alteração, **os dois emitirão o mesmo som genérico** – o que não faz sentido, já que cachorro late e gato mia.

Precisamos de uma forma de cada subclasse **substituir** esse comportamento herdado pelo seu próprio som.

Sobrescrita de métodos: `emitirSom()`

- Queremos que cada tipo de animal emita um som diferente.
- Podemos declarar um método em `Animal` e sobrescrevê-lo (`@Override`) em cada subclasse.
- Isso é diferente de `toString()`: aqui, cada subclasse terá um comportamento totalmente próprio.

Definição: Sobrescrita de Método (*Override*)

Sobrescrita (*Override*)

Ocorre quando uma subclasse fornece uma **nova implementação** para um método já definido na superclasse, mantendo a mesma assinatura (nome, parâmetros e retorno).

- A anotação `@Override` não é obrigatória, mas pede ao compilador para checar se realmente existe um método correspondente na superclasse.
- Evita erros bobos, como escrever o nome do método errado.

Exemplo: Animal com emitirSom() genérico

```
public class Animal {  
    private String nome;  
    private int idade;  
  
    public Animal(String nome, int idade) {  
        this.nome = nome;  
        this.idade = idade;  
    }  
  
    public String getNome() { return nome; }  
    public int getIdade() { return idade; }  
  
    public String emitirSom() {  
        return "Som generico de animal";  
    }  
  
    @Override  
    public String toString() {  
        return "Animal: " + nome + " | Idade: " + idade;  
    }  
}
```

Exemplo: Cachorro sobrescrevendo emitirSom()

```
public class Cachorro extends Animal {
    private String raca;

    public Cachorro(String nome, int idade, String raca) {
        super(nome, idade);
        this.raca = raca;
    }

    public String getRaca() { return raca; }

    @Override
    public String emitirSom() {
        return "Au au!";
    }

    @Override
    public String toString() {
        return super.toString() + " | Raca: " + raca;
    }
}
```

Exemplo: Gato extends Animal

```
public class Gato extends Animal {
    private boolean castrado;

    public Gato(String nome, int idade, boolean castrado) {
        super(nome, idade);
        this.castrado = castrado;
    }

    public boolean isCastrado() { return castrado; }

    @Override
    public String emitirSom() {
        return "Miau!";
    }

    @Override
    public String toString() {
        return super.toString() + " | Castrado: " + castrado;
    }
}
```

Exemplo: Passaro extends Animal

```
public class Passaro extends Animal {
    private boolean podeVoar;

    public Passaro(String nome, int idade, boolean podeVoar) {
        super(nome, idade);
        this.podeVoar = podeVoar;
    }

    public boolean isPodeVoar() { return podeVoar; }

    @Override
    public String emitirSom() {
        return "Piu piu!";
    }

    @Override
    public String toString() {
        return super.toString() + " | Pode voar: " + podeVoar;
    }
}
```

Duas formas de criar um Cachorro

Até agora, sempre criamos objetos assim:

```
Cachorro c1 = new Cachorro("Rex", 3, "Labrador");
```

Mas existe uma outra forma de fazer a mesma criação:

```
Animal a1 = new Cachorro("Rex", 3, "Labrador");
```

Pergunta

As duas linhas criam um Cachorro. Qual é a diferença entre elas?

O tipo do objeto e o tipo da referência

Tipo do objeto (o que realmente existe)

Em `new Cachorro(...)`, o objeto criado na memória é **sempre um Cachorro**, não importa como a variável foi declarada.

Tipo da referência (como enxergamos o objeto)

A variável à esquerda do `=` define **o tipo pelo qual vamos enxergar** esse objeto – pode ser o tipo exato (`Cachorro`) ou um tipo mais genérico (`Animal`).

`c1` e `a1` apontam para o mesmo tipo de objeto, mas cada variável "enxerga" esse objeto de um jeito diferente.

Por que isso é permitido?

- `Cachorro` extends `Animal` significa que todo `Cachorro` **também é um** `Animal`.
- Por isso, é seguro guardar um `Cachorro` dentro de uma variável do tipo `Animal`.

Analogia

Todo cachorro é um animal, mas nem todo animal é um cachorro. Podemos guardar um cachorro numa "caixa" rotulada `Animal`, mas não o contrário.

Pelo mesmo motivo, `Animal a2 = new Gato(...)` também é válido: um `Gato` também é um `Animal`.

O que muda na prática: o que a variável enxerga

Quando a variável é do tipo `Animal`, só é possível acessar o que está declarado em `Animal`:

```
Animal a1 = new Cachorro("Rex", 3, "Labrador");  
  
a1.emitirSom();    // ok, emitirSom existe em Animal  
a1.getNome();      // ok, getNome existe em Animal  
  
a1.getRaca();      // ERRO! getRaca nao existe em Animal
```

Regra

A variável só permite chamar os métodos que existem no seu **próprio tipo declarado**, mesmo que o objeto real tenha outros métodos além desses.

Mas qual som é emitido?

```
Animal a1 = new Cachorro("Rex", 3, "Labrador");  
Animal a2 = new Gato("Mimi", 2, true);  
  
System.out.println(a1.emitirSom()); // "Au au!"  
System.out.println(a2.emitirSom()); // "Miau!"
```

Observação

Mesmo com a1 e a2 declarados como `Animal`, cada um emite o som do seu tipo real (`Cachorro` ou `Gato`), e não o som genérico de `Animal`.

Isso acontece porque o Java sempre executa a versão **sobrescrita** do método, de acordo com o tipo real do objeto – e esse comportamento terá um nome especial, que veremos a seguir.

Uma terceira raça: Golden extends Cachorro

Imagine que agora criamos uma classe ainda mais específica:

```
public class Golden extends Cachorro {  
    // Golden e um tipo específico de Cachorro  
}
```

Agora temos uma cadeia de herança:

Animal ← Cachorro ← Golden

O que isso significa

Todo Golden é um Cachorro, e todo Cachorro é um Animal. Logo, todo Golden **também** é um Animal.

Quais declarações são permitidas?

Com `Golden extends Cachorro extends Animal`, um objeto `Golden` pode ser referenciado de três formas diferentes:

```
Animal a1    = new Golden("Rex", 3, "Labrador"); // ok
Cachorro c1  = new Golden("Rex", 3, "Labrador"); // ok
Golden g1    = new Golden("Rex", 3, "Labrador"); // ok
```

Regra geral

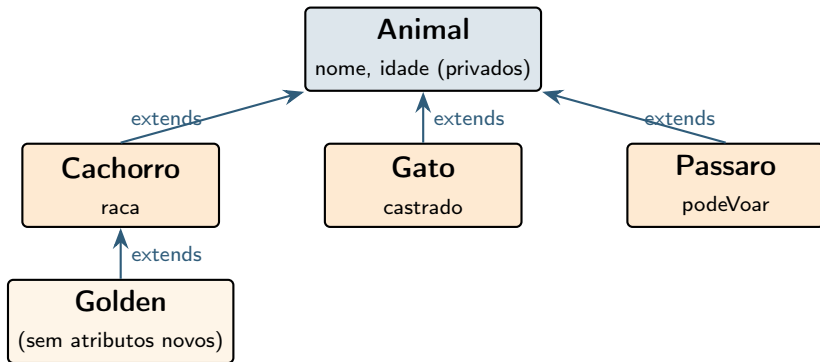
Uma variável pode referenciar um objeto se o tipo da variável for **o mesmo tipo do objeto ou algum de seus ancestrais** (uma classe que está acima dela na cadeia de herança).

Por isso **não** é possível fazer o caminho inverso:

```
Golden g2 = new Cachorro("Rex", 3, "Labrador"); // ERRO!
```

Um `Cachorro` qualquer não é necessariamente um `Golden` – a relação só vale de baixo para cima.

Hierarquia visual: Animal



Cada subclasse herda nome e idade, e sobrescreve `emitirSom()` à sua maneira. Já **Golden** herda tudo de **Cachorro** – inclusive o que **Cachorro** já herdou de **Animal**.

Golden também pode sobrescrever emitirSom()

- Golden herda emitirSom() de Cachorro, que já retorna "Au au!".
- Mas nada impede que Golden sobrescreva esse método novamente, com um latido mais específico.

```
public class Golden extends Cachorro {  
  
    public Golden(String nome, int idade, String raca) {  
        super(nome, idade, raca);  
    }  
  
    @Override  
    public String emitirSom() {  
        return "Au au (latido de Golden)!";  
    }  
}
```

Resultado

```
Golden g1 = new Golden(...);  
g1.emitirSom() → "Au au (latido de Golden)!"
```

Sobrescrever não é exclusivo da superclasse direta

Regra geral

Uma subclasse pode sobrescrever **qualquer método herdado**, seja ele definido na superclasse direta ou em qualquer ancestral mais acima na hierarquia.

No nosso exemplo:

- Cachorro sobrescreve `emitirSom()`, que foi definido em `Animal`;
- Golden sobrescreve `emitirSom()` de novo, mesmo esse método já tendo sido sobrescrito por Cachorro;
- Golden também poderia sobrescrever `toString()`, que foi definido originalmente em `Animal` e já foi sobrescrito por Cachorro.

Ideia-chave

A cada sobrescrita, a subclasse substitui a implementação mais próxima que ela herdou – não importa em qual nível da hierarquia essa implementação foi definida.

Sobrescrever não é exclusivo da superclasse direta

Regra geral

Uma subclasse pode sobrescrever **qualquer método herdado**, mesmo que ele já tenha sido sobrescrito por uma classe intermediária.

Exemplo: quem definiu emitirSom()?

- Animal define a primeira versão: "Som generico de animal"
- Cachorro sobrescreve: "Au au!"
- Golden sobrescreve de novo: "Au au (latido de Golden)!"

Cada subclasse pode escrever sua própria versão, não importa quantas vezes o método já tenha sido sobrescrito antes dela.

Qual versão é executada?

O Java sempre executa a **versão mais próxima do tipo real do objeto**, subindo a hierarquia só até encontrar uma implementação.

```
Animal a1      = new Animal("Bicho", 1);
Cachorro c1    = new Cachorro("Rex", 3, "Vira-lata");
Golden g1      = new Golden("Rex", 3, "Labrador");

a1.emitirSom(); // "Som generico de animal" (definido em Animal)
c1.emitirSom(); // "Au au!"                (sobrescrito em
    Cachorro)
g1.emitirSom(); // "Au au (latido de Golden)!" (sobrescrito em
    Golden)
```

Pense assim

É como um bilhete passado de mão em mão: cada pessoa pode reescrevê-lo do seu jeito. Quando alguém lê o bilhete, vê a **versão mais recente**, não a original.

Uma pergunta importante: o que Golden sobrescreveria?

A dúvida

Se Golden sobrescrever `getNome()`, ele está substituindo a versão de Cachorro ou a versão de `Animal`?

A resposta

Cachorro **nunca sobrescreveu** `getNome()`. Isso significa que não existe uma "versão de Cachorro" para esse método – existe apenas **uma única implementação**, definida em `Animal`.

Quando Golden sobrescreve `getNome()`, ele está substituindo essa única implementação, que veio de `Animal`, mesmo passando "por dentro" de Cachorro na hierarquia.

Comparando emitirSom() e getNome()

- emitirSom(): **foi sobrescrito em todos os níveis** – Animal, Cachorro e Golden têm, cada um, sua própria versão.
- getNome(): **só existe uma versão**, a de Animal – Cachorro apenas herdou, sem alterar.

```
public class Golden extends Cachorro {  
    @Override  
    public String getNome() {  
        // super.getNome() sobe ate encontrar uma  
        // implementacao: como Cachorro nao tem uma  
        // propria, quem responde e Animal.  
        return "Golden " + super.getNome();  
    }  
}
```

Ideia-chave

Ao sobrescrever, a subclasse substitui **a implementação mais próxima que existe na cadeia de herança** – não importa em qual classe ela foi definida, nem quantos níveis o método "atravessou" sem ser alterado.

Por que não chamar set dentro do construtor?

O problema

Um método set público pode ser **sobrescrito** por uma subclasse. Se o construtor da superclasse chama esse set, ele pode acabar executando a versão sobrescrita da subclasse – antes mesmo dela existir por completo.

Ordem real de construção em Java

- 1 o construtor da superclasse é executado (`super(...)`);
- 2 os atributos da subclasse são inicializados;
- 3 o corpo do construtor da subclasse é executado.

Se o passo 1 chamar um método sobrescrito no passo 2 ainda não aconteceu, o método roda com atributos da subclasse **ainda não inicializados**.

Vendo o bug acontecer

```
public class Animal {
    private String nome;
    public Animal(String nome) {
        setNome(nome); // chama um metodo sobrescrevivel!
    }
    public void setNome(String nome) {
        this.nome = nome;
    }
}

public class Cachorro extends Animal {
    private List<String> historico;
    public Cachorro(String nome) {
        super(nome); // Animal() chama setNome() aqui dentro
        historico = new ArrayList<>(); // so inicializa AQUI
    }
    @Override
    public void setNome(String nome) {
        historico.add(nome); // historico ainda e null!
        super.setNome(nome);
    }
}
```

O que aconteceu?

Resultado

`new Cachorro("Rex")` lança `NullPointerException`.

- `super(nome)` executa o construtor de `Animal`, que chama `setNome(nome)`;
- como `setNome()` foi sobrescrito, quem executa é a versão de `Cachorro`;
- essa versão tenta usar `historico` – mas ele **ainda não foi inicializado**, pois isso só acontece **depois** que `super(nome)` terminar.

Resumo

O método sobrescrito é executado **antes** de `Cachorro` terminar de se construir.

A solução: validação em método privado

Ideia

Métodos **privados** não podem ser sobrescritos – logo, chamá-los no construtor é sempre seguro. O construtor e o set passam a compartilhar a mesma validação, sem passar um pelo outro.

```
public class Animal {
    private String nome;

    public Animal(String nome) {
        this.nome = validarNome(nome);
    }

    public void setNome(String nome) {
        this.nome = validarNome(nome);
    }

    private String validarNome(String nome) {
        if (nome == null || nome.isBlank()) {
            throw new IllegalArgumentException("Nome invalido");
        }
        return nome;
    }
}
```

Parte 3 – Polimorfismo

Um objeto, muitos comportamentos

Polimorfismo: tratando objetos diferentes de forma uniforme

- **Poli** = muitas, **morfo** = formas → "muitas formas".
- Podemos usar uma referência do tipo `Animal` para guardar um Cachorro, um Gato ou um Passaro.
- Ao chamar um método sobrescrito (como `emitirSom()`), o Java executa a versão da **classe real** do objeto, não da referência.
- Essa decisão acontece em tempo de execução – chamamos isso de *ligação tardia*.

Definição: Polimorfismo

Polimorfismo

É a capacidade de um mesmo método se comportar de maneiras diferentes, dependendo do objeto que o executa, permitindo tratar objetos de subclasses distintas por meio de uma **referência comum** da superclasse.

- Em Java, o polimorfismo de sobrescrita é resolvido **em tempo de execução**.
- É um dos pilares da orientação a objetos, junto com encapsulamento e herança.

Exemplo: ArrayList de Animal (polimorfismo)

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList<Animal> animais = new ArrayList<Animal>();

        animais.add(new Cachorro("Rex", 3, "Labrador"));
        animais.add(new Gato("Mimi", 2, true));
        animais.add(new Passaro("Piu", 1, true));

        for (Animal animal : animais) {
            System.out.println(animal.getNome() +
                               " diz: " + animal.emitirSom());
        }
    }
}
```

Exemplo: saída do ArrayList polimórfico

```
Rex diz: Au au!  
Mimi diz: Miau!  
Piu diz: Piu piu!
```

- O ArrayList é declarado como ArrayList<Animal>, mas cada posição guarda um objeto de uma subclasse diferente.
- animal.emitirSom() chama a versão correta automaticamente, conforme o tipo real do objeto.

Pense um pouco

Se a classe Cachorro **não** sobrescrevesse emitirSom(), qual versão do método seria executada ao chamar `rex.emitirSom()`?

- Dica: pense em como o Java procura o método – primeiro na classe do objeto, depois nas superclasses.

Por que isso é útil?

- Podemos escrever um laço genérico que funciona para qualquer `Animal`, atual ou futuro.
- Se criarmos uma nova subclasse (ex.: `Peixe`), o mesmo laço continua funcionando, sem precisar mudar o `Main`.
- Isso deixa o código mais flexível, organizado e fácil de estender.

Em poucas palavras

- Uma referência de superclasse pode apontar para qualquer subclasse.
- O método executado é sempre o da **classe real** do objeto.
- Isso permite escrever código genérico (laços, ArrayList) que funciona com várias subclasses.

Parte 4 – Classes Abstratas

Obrigando as subclasses a fazerem a parte delas

Problema: Animal genérico não faz muito sentido

- O que `emitirSom()` deveria retornar para um `Animal` "genérico", sem espécie definida?
- Também não faz muito sentido criar um objeto `Animal` puro, sem ele ser de uma espécie específica.
- Queremos impedir isso e **obrigar** cada subclasse a definir seu próprio `som`.

A solução: classes abstratas

- Uma classe **abstrata** representa um conceito geral que não deve ser instanciado sozinho.
- Pode ter atributos, construtor e métodos normais (concretos), como qualquer classe.
- Pode ter **métodos abstratos**: sem corpo, que as subclasses são **obrigadas** a implementar.

Definição: Classe Abstrata

Classe abstrata

É uma classe que **não pode ser instanciada diretamente**, usada para representar um conceito geral e compartilhar código entre suas subclasses. Pode conter métodos concretos e métodos abstratos.

- Palavra-chave em Java: `abstract class`.
- Serve como um "molde" comum para as subclasses concretas.

Definição: Método Abstrato

Método abstrato

É um método declarado **sem implementação** (sem corpo), que obriga toda subclasse concreta a fornecer sua própria implementação.

- Só pode existir dentro de uma classe abstract.
- Termina com ;, em vez de { }.

Sintaxe: abstract class e método abstrato

```
public abstract class Superclasse {  
    public abstract TipoRetorno metodoAbstrato();  
}
```

- A classe é marcada com `abstract` antes de `class`.
- O método abstrato termina com `;` em vez de `{ }` – ele não tem corpo.
- Uma classe com pelo menos um método abstrato **precisa** ser abstrata.

Exemplo: Animal abstrato

```
public abstract class Animal {  
    private String nome;  
    private int idade;  
  
    public Animal(String nome, int idade) {  
        this.nome = nome;  
        this.idade = idade;  
    }  
  
    public String getNome() { return nome; }  
    public int getIdade() { return idade; }  
  
    public abstract String emitirSom();  
  
    @Override  
    public String toString() {  
        return "Animal: " + nome + " | Idade: " + idade;  
    }  
}
```

Atenção: não dá para instanciar uma classe abstrata

```
public class Main {  
    public static void main(String[] args) {  
        Animal a = new Animal("Rex", 3);  
        // ERRO DE COMPILACAO:  
        // Cannot instantiate the type Animal  
    }  
}
```

Atenção

- O compilador impede isso porque a classe está incompleta (tem um método sem implementação).
- Só podemos instanciar as **subclasses concretas** (Cachorro, Gato, Passaro), que implementam emitirSom().

As subclasses continuam iguais – agora com uma obrigação

- As classes Cachorro, Gato e Passaro não mudam: continuam com `extends Animal` e `@Override emitirSom()`.
- A diferença é que, agora, sobrescrever `emitirSom()` **deixa de ser opcional**.
- Se você apagar o `emitirSom()` de Cachorro, o código simplesmente não compila mais.
- O compilador garante que toda subclasse concreta saiba "emitir seu som".

Exemplo: o mesmo Main, agora com Animal abstrato

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList<Animal> animais = new ArrayList<Animal>();

        animais.add(new Cachorro("Rex", 3, "Labrador"));
        animais.add(new Gato("Mimi", 2, true));
        animais.add(new Passaro("Piu", 1, true));

        for (Animal animal : animais) {
            System.out.println(animal.getNome() +
                               " diz: " + animal.emitirSom());
        }
    }
}
```

A saída é exatamente a mesma de antes – o polimorfismo continua funcionando, agora com a garantia da classe abstrata.

Pense um pouco

Por que não bastaria deixar `emitirSom()` em `Animal` com um corpo vazio (`return ;`), em vez de torná-lo abstrato?

- Dica: pense no que acontece se uma subclasse nova **esquecer** de sobrescrever o método.

Quando usar instanceof?

- Às vezes uma subclasse tem um método **próprio**, que não existe na superclasse.
- Para chamá-lo a partir de uma referência da superclasse, precisamos primeiro verificar o tipo real do objeto.
- O operador `instanceof` verifica esse tipo antes de fazer um *cast* (conversão) com segurança.
- Use com moderação: depender muito de `instanceof` costuma indicar que falta um método abstrato bem pensado na superclasse.

Exemplo: instanceof e *downcasting*

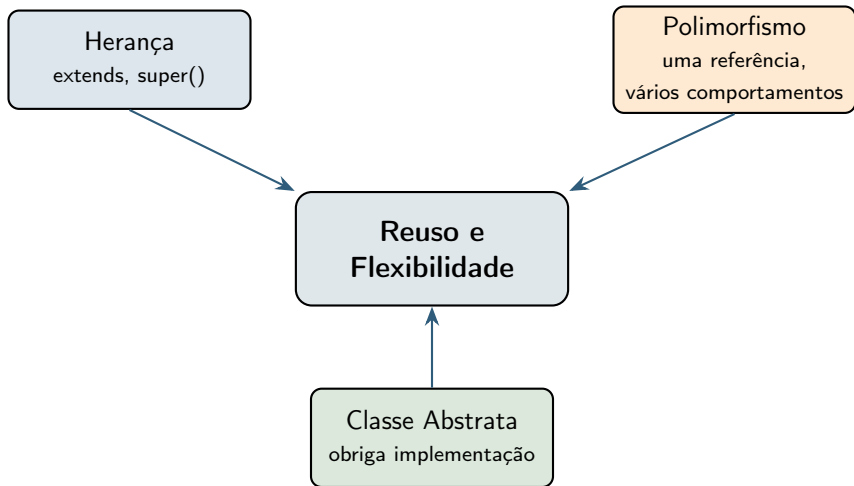
```
for (Animal animal : animais) {  
    if (animal instanceof Passaro) {  
        Passaro p = (Passaro) animal;  
        System.out.println(p.getNome() +  
            " pode voar: " + p.isPodeVoar());  
    }  
}
```

- `isPodeVoar()` só existe em `Passaro`, não em `Animal`.
- O `if` garante que o *cast* `(Passaro) animal` só acontece quando é seguro.

Em poucas palavras

- `abstract class`: não pode ser instanciada, mas pode ter atributos, construtor e métodos concretos.
- `abstract method`: sem corpo, obriga as subclasses concretas a implementá-lo.
- Combina muito bem com polimorfismo: garante que todo objeto da hierarquia "sabe se comportar".

Resumo Visual: os três conceitos juntos



Os três conceitos trabalham juntos: a herança organiza, o polimorfismo flexibiliza, e a classe abstrata garante que cada subclasse cumpra seu papel.

Resumo: Herança, Polimorfismo e Classes Abstratas

- **Herança** (`extends`) evita duplicação: atributos e métodos comuns ficam na superclasse.
- Atributos privados da superclasse não são acessados diretamente; usamos `super(...)` e *getters*.
- `@Override` permite que cada subclasse personalize um comportamento; `super.metodo()` reaproveita o da superclasse.
- **Polimorfismo**: uma referência da superclasse pode apontar para qualquer subclasse, e o método correto é escolhido em tempo de execução.
- **Classes abstratas** impedem instâncias "genéricas" e obrigam as subclasses a implementar métodos essenciais.
- Juntos, esses três conceitos tornam o código mais organizado, reaproveitável e fácil de estender.

Parte 5 – Prática Guiada

Colocando a mão na massa

Problema:

- Crie uma classe abstrata `Forma` com:
 - atributo privado `nome`;
 - construtor e *getter*;
 - método abstrato `calcularArea()`;
 - `toString()` concreto, usando `calcularArea()`.
- Crie as subclasses `Circulo` (com raio) e `Retangulo` (com largura e altura).
- No `Main`, crie um `ArrayList` de `Forma`, imprima cada uma e some todas as áreas.

Gabarito: Classe abstrata Forma

```
public abstract class Forma {  
    private String nome;  
  
    public Forma(String nome) {  
        this.nome = nome;  
    }  
  
    public String getNome() { return nome; }  
  
    public abstract double calcularArea();  
  
    @Override  
    public String toString() {  
        return nome + " | Area: " + calcularArea();  
    }  
}
```

Gabarito: Classe Circulo

```
public class Circulo extends Forma {  
    private double raio;  
  
    public Circulo(double raio) {  
        super("Circulo");  
        this.raio = raio;  
    }  
  
    public double getRaio() { return raio; }  
  
    @Override  
    public double calcularArea() {  
        return Math.PI * raio * raio;  
    }  
}
```

Gabarito: Classe Retangulo

```
public class Retangulo extends Forma {
    private double largura;
    private double altura;

    public Retangulo(double largura, double altura) {
        super("Retangulo");
        this.largura = largura;
        this.altura = altura;
    }

    public double getLargura() { return largura; }
    public double getAltura() { return altura; }

    @Override
    public double calcularArea() {
        return largura * altura;
    }
}
```

Gabarito: Main com polimorfismo de formas

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList<Forma> formas = new ArrayList<Forma>();

        formas.add(new Circulo(5.0));
        formas.add(new Retangulo(4.0, 6.0));

        double total = 0;

        for (Forma forma : formas) {
            System.out.println(forma);
            total += forma.calcularArea();
        }

        System.out.println("Area total: " + total);
    }
}
```

Saída esperada (formas)

```
Circulo | Area: 78.53981633974483  
Retangulo | Area: 24.0  
Area total: 102.53981633974483
```

O laço não sabe (nem precisa saber) se está lidando com um `Circulo` ou um `Retangulo` – ele só chama `calcularArea()` e confia no polimorfismo.

Problema:

- Crie uma classe abstrata `Veiculo` com:
 - atributos privados `modelo` e `velocidadeMaxima`;
 - construtor e *getters*;
 - método abstrato `tipoDeMotor()`;
 - `toString()` concreto, usando `tipoDeMotor()`.
- Crie as subclasses `Carro` (com portas) e `Moto` (com `temBau`).
- No `Main`, crie um `ArrayList` de `Veiculo` e imprima todos, usando polimorfismo.

Gabarito: Classe abstrata Veiculo

```
public abstract class Veiculo {
    private String modelo;
    private double velocidadeMaxima;

    public Veiculo(String modelo, double velocidadeMaxima) {
        this.modelo = modelo;
        this.velocidadeMaxima = velocidadeMaxima;
    }

    public String getModelo() { return modelo; }
    public double getVelocidadeMaxima() { return velocidadeMaxima; }

    public abstract String tipoDeMotor();

    @Override
    public String toString() {
        return modelo + " | Vel. Max: " + velocidadeMaxima +
            " km/h | Motor: " + tipoDeMotor();
    }
}
```

Gabarito: Classe Carro

```
public class Carro extends Veiculo {
    private int portas;

    public Carro(String modelo, double velocidadeMaxima, int portas) {
        super(modelo, velocidadeMaxima);
        this.portas = portas;
    }

    public int getPortas() { return portas; }

    @Override
    public String tipoDeMotor() {
        return "Motor a combustao (4 tempos)";
    }
}
```

```
public class Moto extends Veiculo {
    private boolean temBau;

    public Moto(String modelo, double velocidadeMaxima, boolean
        temBau) {
        super(modelo, velocidadeMaxima);
        this.temBau = temBau;
    }

    public boolean isTemBau() { return temBau; }

    @Override
    public String tipoDeMotor() {
        return "Motor de duas rodas";
    }
}
```

Gabarito: Main com polimorfismo de veículos

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList<Veiculo> veiculos = new ArrayList<Veiculo>();

        veiculos.add(new Carro("Civic", 210.0, 4));
        veiculos.add(new Moto("CB500", 180.0, false));

        for (Veiculo veiculo : veiculos) {
            System.out.println(veiculo);
        }
    }
}
```

Saída esperada (veículos)

```
Civic | Vel. Max: 210.0 km/h | Motor: Motor a combustao (4 tempos)
CB500 | Vel. Max: 180.0 km/h | Motor: Motor de duas rodas
```

De novo: o Main trata Carro e Moto apenas como Veiculo – e cada um responde com seu próprio tipoDeMotor().

Parte 6 – Desafio Final

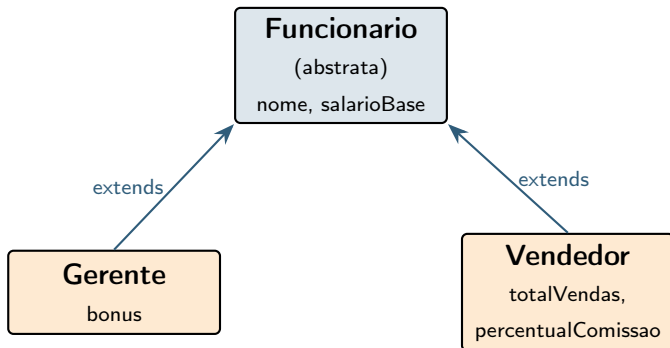
Agora é sua vez de generalizar uma hierarquia

Desafio final: Sistema de Folha de Pagamento

Sua missão:

- 1 Crie uma classe abstrata `Funcionario`, com atributos **privados** `nome` e `salarioBase`.
- 2 Adicione um construtor, *getters* e um método abstrato `calcularSalario()`.
- 3 Crie a subclasse `Gerente`, que recebe um `bonus` fixo somado ao salário base.
- 4 Crie a subclasse `Vendedor`, cujo salário depende do total de vendas e de um percentual de comissão.
- 5 Sobrescreva `calcularSalario()` de forma diferente em cada subclasse.
- 6 Crie também `toString()` em `Funcionario`, reaproveitando `calcularSalario()`.
- 7 No Main, crie um `ArrayList` de `Funcionario` com `Gerentes` e `Vendedores`, e imprima a folha de pagamento completa, usando polimorfismo.

Desafio Visual: Folha de Pagamento



Cada subclasse implementa `calcularSalario()` à sua maneira – e o Main consegue somar a folha inteira sem nunca verificar "se é Gerente ou Vendedor".